Universidad — Programación Avanzada

Práctica de Socket con Hilos

Servidor multihilo con mensajería entre clientes

Juan Tartaglia

Legajo 115387

Programación Avanzada — 2026

1. Introducción

Esta práctica extiende la implementación de sockets TCP de la entrega anterior incorporando el concepto de **concurrencia mediante hilos**. El objetivo es que el servidor pueda atender a múltiples clientes de forma simultánea, permitiendo además la **comunicación entre ellos** a través del servidor como intermediario.

Cada cliente que se conecta recibe un **nombre de usuario único** y un menú de comandos. Puede consultar la fecha y hora, resolver expresiones matemáticas, listar los usuarios conectados y enviar mensajes privados o globales. El servidor registra en su consola todo lo que ocurre en tiempo real.

2. Arquitectura de la solución

El servidor mantiene un **ConcurrentHashMap** que asocia cada nombre de usuario con su *ClienteHandler* correspondiente. Este mapa es accedido por todos los hilos, por lo que el uso de una estructura concurrente garantiza la seguridad en el acceso simultáneo sin necesidad de bloqueos explícitos.

Clase Servidor

El método *main* abre un *ServerSocket* en el puerto 5000 y entra en un bucle infinito. Por cada cliente aceptado instancia un *ClienteHandler* y lo lanza en un nuevo hilo daemon. Los métodos estáticos *broadcast()*, *enviarA()*, *registrarCliente()* y *desregistrarCliente()* son compartidos por todos los hilos y operan sobre el mapa concurrente.

Clase ClienteHandler

Implementa *Runnable*. Al iniciarse solicita el nombre de usuario, lo registra de forma sincronizada para garantizar unicidad, y notifica a los demás clientes la nueva conexión. Luego entra en un bucle leyendo comandos y generando respuestas. Al finalizar (por SALIR o pérdida de conexión) desregistra al cliente y notifica al resto.

Clase Cliente

Se conecta al servidor y lanza un **hilo lector daemon** que muestra los mensajes entrantes en tiempo real. El hilo principal lee la entrada del usuario y la envía al servidor. Esta arquitectura permite recibir mensajes de otros clientes mientras se escribe, sin bloqueos.

3. Protocolo y comandos

La comunicación usa texto plano con líneas terminadas en *\n*. El conjunto de comandos disponibles es el siguiente:

```
AYUDA                  Muestra el menu de comandos
FECHA                  Retorna la fecha y hora actual del servidor
RESOLVE <expr>         Evalua una expresion matematica
LISTA                  Lista los usuarios conectados en ese momento
*<usuario> <mensaje>   Envia un mensaje privado a un usuario especifico
```

```
*ALL <mensaje>          Envia un mensaje a todos los clientes conectados
SALIR                   Cierra la conexion del cliente
```

El servidor valida cada comando y maneja los errores de forma informativa. Por ejemplo, si el destinatario de un mensaje privado no existe, responde al remitente indicando el error y la lista actualizada de usuarios conectados.

4. Asignación de nombres únicos

El método *asignarNombre()* está declarado **synchronized** para evitar condiciones de carrera cuando dos clientes se conectan casi simultáneamente con el mismo nombre. Si el nombre solicitado ya está en uso, el servidor agrega un sufijo numérico incremental (Juan, Juan2, Juan3...) e informa al cliente del cambio en el mensaje de bienvenida.

5. Estructura del proyecto en NetBeans

El proyecto **ClienteServidorHilos** mantiene la misma estructura que la práctica anterior: dos clases en el paquete por defecto, cada una con su método *main* para poder ejecutarlas de forma independiente.

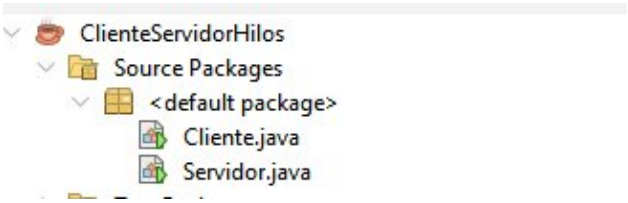

Figura 1 — Estructura del proyecto en el panel Projects de NetBeans.

6. Ejecución y capturas de pantalla

La sesión de prueba se realizó con dos clientes simultáneos: **Juan** y **Maria**. Se ejecutó primero el Servidor, luego dos instancias del Cliente mediante *Run File* en NetBeans. A continuación se documenta la sesión completa.

6.1 Log del Servidor

El servidor registra cada evento con timestamp: conexiones, comandos recibidos, mensajes enrutados y desconexiones.

```
ClienteServidorHilos (run) ×   ClienteServidorHilos (run) #2 ×   ClienteServidorHilos (run) #3 ×
run:
[2026-04-16 16:40:43] [SISTEMA] Servidor iniciando en puerto 5000
[2026-04-16 16:40:43] [SISTEMA] Esperando conexiones...
[2026-04-16 16:41:13] [SISTEMA] Cliente registrado: Juan | Conectados: 1
[2026-04-16 16:41:28] [SISTEMA] Cliente registrado: Maria | Conectados: 2
[2026-04-16 16:42:17] [Juan] FECHA
[2026-04-16 16:42:29] [Juan] RESOLVE 45*23/54+234
[2026-04-16 16:42:39] [Juan] LISTA
[2026-04-16 16:42:50] [Juan] *Maria Hola!
[2026-04-16 16:42:50] [Juan] Mensaje privado a Maria: Hola!
[2026-04-16 16:43:00] [Juan] *ALL Como estan todos?
[2026-04-16 16:43:00] [Juan] Broadcast: Como estan todos?
[2026-04-16 16:43:16] [Maria] *Juan Hola Juan!
[2026-04-16 16:43:16] [Maria] Mensaje privado a Juan: Hola Juan!
[2026-04-16 16:43:36] [Maria] *UsuarioInexistente Hola
[2026-04-16 16:43:36] [SERVIDOR] Maria intento enviar a 'UsuarioInexistente' (no existe).
[2026-04-16 16:43:58] [Juan]  SALIR
[2026-04-16 16:43:58] [SISTEMA] Cliente desconectado: Juan | Conectados: 1
[2026-04-16 16:44:19] [Maria]  SALIR
[2026-04-16 16:44:19] [SISTEMA] Cliente desconectado: Maria | Conectados: 0
```

Figura 2 — Consola del Servidor mostrando el log completo de la sesión con dos clientes conectados simultáneamente.

6.2 Consola del Cliente Juan

Juan recibe la bienvenida, consulta la fecha, resuelve una expresión matemática, lista los usuarios, envía un mensaje privado a Maria, un broadcast a todos, recibe la respuesta de Maria y finaliza la sesión.

```
ClienteServidorHilos (run) ×   ClienteServidorHilos (run) #2 ×   ClienteServidorHilos (run) #3 ×
[CLIENTE] Conectando a localhost:5000...
[CLIENTE] Conexion establecida.

>
[SERVIDOR] Ingresa tu nombre de usuario:
> Juan
>
[SERVIDOR] ---
>
[SERVIDOR] Bienvenido, Juan!
>
[SERVIDOR]   COMANDOS DISPONIBLES
>
[SERVIDOR]   AYUDA                  - Muestra este menu
>
[SERVIDOR]   FECHA                  - Fecha y hora actual
>
[SERVIDOR]   RESOLVE <expr>         - Resuelve expresion matematica
>
[SERVIDOR]   LISTA                  - Lista de clientes conectados
>
[SERVIDOR]   *<usuario> <mensaje>   - Envia mensaje privado
>
[SERVIDOR]   *ALL <mensaje>         - Envia mensaje a todos
>
[SERVIDOR]   SALIR                  - Desconectarse
>
[SERVIDOR] ---
>
[SERVIDOR] [SERVIDOR] Maria se ha conectado.
> FECHA
>
[SERVIDOR] Fecha y hora: jueves 16/04/2026 16:42:18
> RESOLVE 45*23/54+234
>
[SERVIDOR] Resultado de [45*23/54+234] = 253.16666666666666
> LISTA
>
[SERVIDOR] Conectados (2): Juan, Maria
> *Maria Hola!
>
[SERVIDOR] [ENVIADO -> Maria] Hola!
> *ALL Como estan todos?
>
[SERVIDOR] [Juan -> TODOS] Como estan todos?
>
[SERVIDOR] Mensaje enviado a todos.
>
[SERVIDOR] [Maria -> Juan] Hola Juan!
>  SALIR

[SERVIDOR] Hasta luego, Juan!
> [CLIENTE] Desconectado.
```

Figura 3 — Consola del Cliente Juan: uso de FECHA, RESOLVE, LISTA, mensajes privados, broadcast y SALIR.

6.3 Consola del Cliente Maria

Maria recibe su bienvenida, los mensajes enviados por Juan, responde con un mensaje privado, intenta enviar a un usuario inexistente (prueba de error) y finaliza la sesión.

```
Output
ClienteServidorHilos (run)   ClienteServidorHilos (run) #2   ClienteServidorHilos (run) #3
run:
[CLIENTE] Conectando a localhost:5000...
[CLIENTE] Conexion establecida.

>
[SERVIDOR] Ingresa tu nombre de usuario:
> Maria
>
[SERVIDOR] ---
>
[SERVIDOR] Bienvenido, Maria!
>
[SERVIDOR]   COMANDOS DISPONIBLES
>
[SERVIDOR]   AYUDA                  - Muestra este menu
>
[SERVIDOR]   FECHA                  - Fecha y hora actual
>
[SERVIDOR]   RESOLVE <expr>         - Resuelve expresion matematica
>
[SERVIDOR]   LISTA                  - Lista de clientes conectados
>
[SERVIDOR]   *<usuario> <mensaje>   - Envia mensaje privado
>
[SERVIDOR]   *ALL <mensaje>         - Envia mensaje a todos
>
[SERVIDOR]   SALIR                  - Desconectarse
>
[SERVIDOR] ---
>
[SERVIDOR] [Juan -> Maria] Hola!
>
[SERVIDOR] [Juan -> TODOS] Como estan todos?
> *Juan Hola Juan!
>
[SERVIDOR] [ENVIADO -> Juan] Hola Juan!
> *UsuarioInexistente Hola
>
[SERVIDOR] ERROR: El usuario 'UsuarioInexistente' no esta conectado. Conectados (2): Juan, Maria
>
[SERVIDOR] [SERVIDOR -> TODOS] Juan se ha desconectado.
>  SALIR

[SERVIDOR] Hasta luego, Maria!
> [CLIENTE] Desconectado.
BUILD SUCCESSFUL (total time: 2 minutes 55 seconds)
```

Figura 4 — Consola del Cliente Maria: recepción de mensajes, respuesta privada, manejo de error por usuario inexistente y SALIR.

7. Análisis de resultados

Todos los requisitos de la consigna fueron cumplidos satisfactoriamente. El servidor atendió a dos clientes en hilos independientes sin interferencias. El comando **FECHA** retornó correctamente la fecha del sistema. El **RESOLVE** evaluó la expresión con precedencia de operadores correcta obteniendo 253.166... . El comando **LISTA** mostró en tiempo real los dos usuarios conectados.

La mensajería funcionó en ambas direcciones: el mensaje privado **Maria Hola!* fue recibido únicamente por Maria, y el broadcast **ALL Como estan todos?* llegó a ambos clientes. Al intentar enviar a **UsuarioInexistente**, el servidor informó el error al remitente con la lista actualizada de conectados, demostrando el manejo de errores solicitado en la consigna.

El cierre de conexión con **SALIR** fue limpio en ambos casos: el servidor desregistró al cliente, notificó a los demás y quedó disponible para nuevas conexiones.

8. Conclusión

La incorporación de hilos al servidor permitió transformar una aplicación de atención secuencial en un sistema de mensajería concurrente funcional. El uso de *ConcurrentHashMap* y métodos sincronizados garantizó la integridad de los datos compartidos entre hilos. La arquitectura resultante es extensible: agregar nuevos comandos o funcionalidades requiere únicamente modificar el método *procesarComando()* del *ClienteHandler*, sin alterar la lógica de concurrencia.